

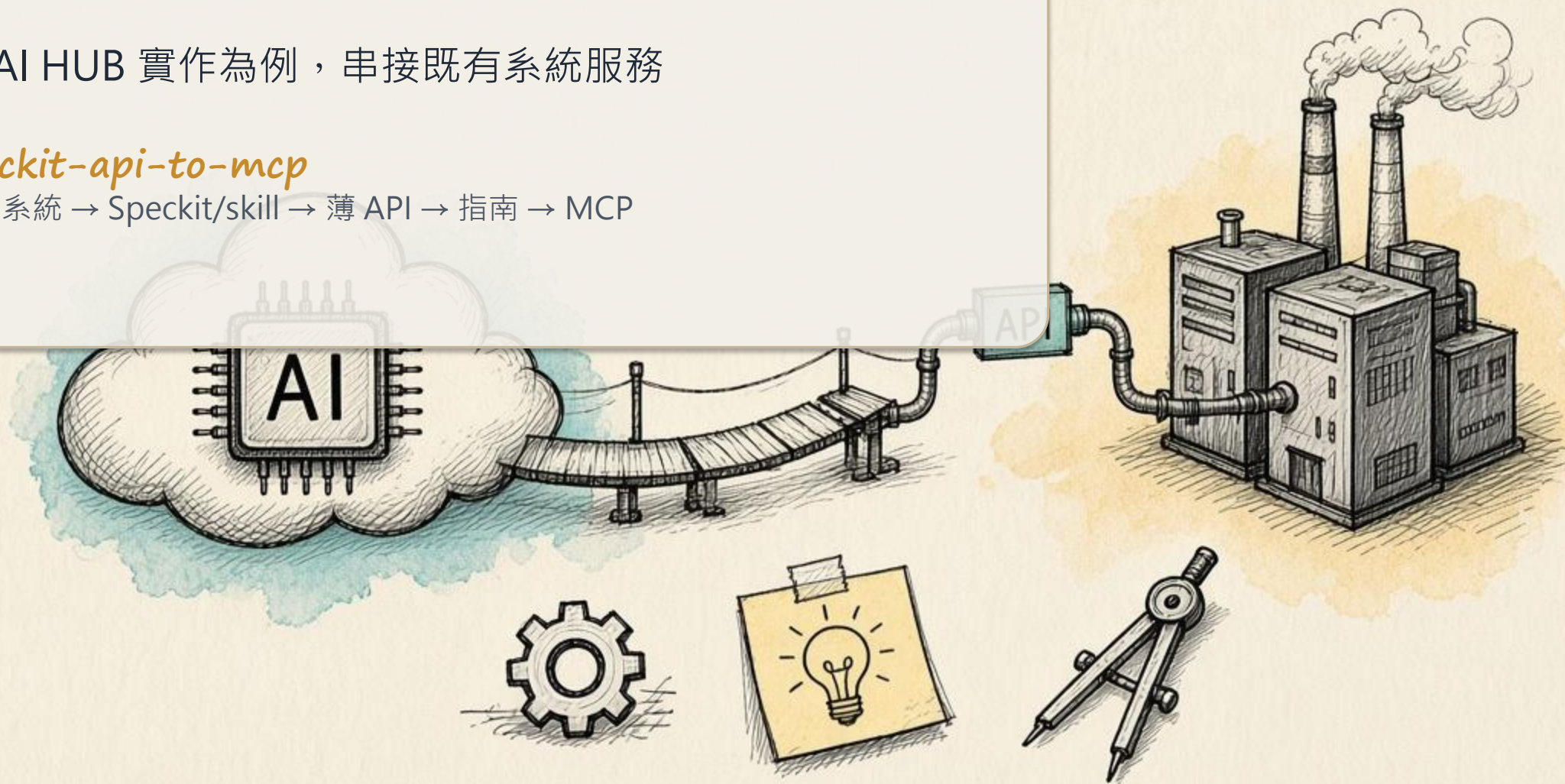
FIELD NOTES | AI HUB

API 到 skill 到 MCP

以 AI HUB 實作為例，串接既有系統服務

speckit-api-to-mcp

既有系統 → Speckit/skill → 薄 API → 指南 → MCP



目錄

01

痛點與目的

為什麼需要這條管線

02

三個重點

Skill → MCP → 授權分層

03

Skill vs MCP

差異、場景、拆分三問

04

管線詳解

階段 0-8

05

開跑、案例、效益

指令、適用與不適用

Agent 會寫程式，但不會「用我們的系統」

1 重複開發

複製貼上越來越亂；API、文件、串接多套版本。

2 Speckit 切入難

系統上線中，不易判斷從哪裡下手最安全。

3 文件難產

接口有了，但缺少人與 Agent 都讀得懂的指南。

4 串接繁瑣

規格一改，呼叫端、MCP、驗證都要手動同步。

建立AI模型協作方法，有效管理程式與文件

Flow

管流程

- 建立SOP
- skill 收成固定管線
- 規格變更可重跑

Quality

管品質

- 先審計再動筆
- 最小變動、新檔才寫
- 禁止無故覆寫

Artifacts

管產物

- HTTP 薄 API
- 呼叫指南 md
- Node.js MCP + dist/

三件事串成一條可複製方法

01

Skill + Speckit

最小變動建 API

既有系統利用 skill，套用 Speckit，薄封裝成標準接口。

02

Skill -> MCP

AI 模型生成

依呼叫指南由模型生成 MCP，掛上 AI HUB / Cursor。

03

Auth Layers

OS / API / MCP

各層只暴露完成任務所需的最小能力。

關鍵差異：SOP 大腦 vs 連接協定

Skill = Brain / SOP

Essence

思維與 SOP 指引

Layer

Prompt / Context

Goal

教模型「如何做」

How

Markdown 指引推論

State

輕量、主要 Stateless

MCP = Hands & Eyes

連接與工具協定

Protocol / Infra

讓模型「連結到」

JSON-RPC Client-Server

進程/Server，可 Stateful

何時用 Skill？何時用 MCP？

Skill scenarios

領域分析與寫作

Code Review 規範、品牌文案

多步驟推論

拆解 → 驗證 → 結論

範本與結構化輸出

Weekly Report、ADR、API 文件

引導式對話

教學、面試模擬、診斷問答

MCP scenarios

外部資料源

GitHub / DB / Notion / Jira

系統操作（副作用）

Terminal、PR、Email、CI/CD

動態 API 整合

即時資料、報價、地理

跨系統串接

A 輸出 → 轉換 → 寫入 B

MCP=手眼；Skill=大腦 SOP

Decision tree

1 有系統副作用或資料存取？

是 → MCP Server；否 → 下一問

2 需要思考順序／檢核／輸出格式？

是 → Skill；否 → 標準 System Prompt

3 先抓資料 → 依邏輯處理 → 寫回？

是 → Skill + MCP 組合

Architecture 怎麼講

腦：Skill 規定「怎麼想、怎麼做」

手眼：MCP 負責「連到哪、呼叫什麼」

外系：DB / 既有 API 只暴露最小能力

一句話：Skill 決策，MCP 執行，系統給資料

Example: Code Review

場景：要幫 PR 做 Code Review

MCP：get_pr_diff / post_comment (連 GitHub)

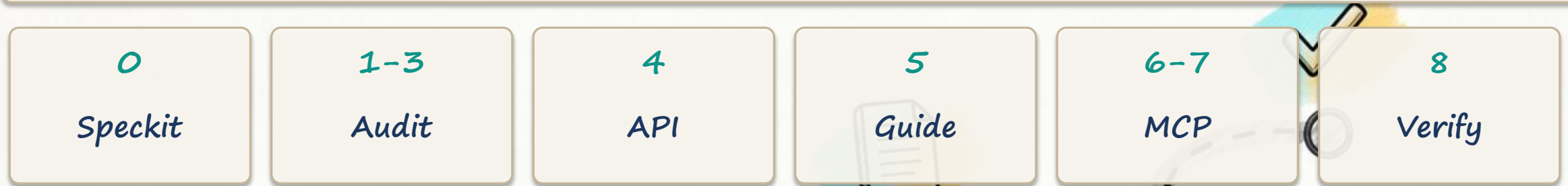
Skill：Style Guide、安全清單、回覆格式

口訣：SOP 變→改 Skill；API 變→改 MCP

切勿：SOP 不寫進 MCP；API 細節不塞進 Skill

04 | Pipeline

拆解痛點，將工作分為 9 個階段



Front half: long-term memory

- 先讀 constitution 與 rules
- 已涵蓋則跳過；部分只補
- 禁止整份覆寫、--force
- Speckit 詳、Rule 精

Back half: three artifacts

- HTTP 薄 API
- 每功能一份指南 md
- Node.js MCP (stdio)
- dist/*.mjs + package.json

階段 0~3：先審計、找安全切入點

管線前半 (階段 0~3)：先讀再寫 → 已涵蓋則跳過 → 部分只補 → 沒有才新建

0

階段 0 Speckit

確認 / 導入；未完成不得進階段 1

1

階段 1 Constitution

技術梭、呼叫 / 回傳格式就緒；已有不重寫

2

階段 2 Architecture Rules

結構、DB、控制器、工具、慣例；精簡不重複

3

階段 3 API Rules

五條約束生效；長期記憶僅缺漏才補

階段 4：五條硬約束（薄封裝，不重構）

1

New files only

新建程式碼獨立新檔，不汙染既有內容。

2

Reuse functions

驗證、紀錄查詢須呼叫現成函式。

3

No DB schema change

不加表不改欄；僅組既有邏輯建 API。

4

Reuse ViewModel

有現成就用，不另起一套模型。

5

Ask before refactor

共通程式有誤 / 慢，不可直接改。

階段 5：呼叫指南—解決文件難產

每功能一份 md；先詢 Base URL (勿假設 port)

1

Basic

基本資訊

Base URL、回應格式

2

Endpoint

端點說明

Method / path / 依賴

3

Params

必要參數

名、型、模式

4

Response

回應欄位

欄位與說明

5

Limits

限制規則

All-or-Nothing、上限

6

Flow

使用流程

步驟說明

7

Notes

注意事項

易錯點與限制

階段 6 - 8 : AI 生成 MCP 並驗證

6 MCP Server

- 讀全部呼叫指南 md
- Node.js + stdio
- 預留 SSE
- 與指南同資料夾
- port 預設 8895

7 Build dist

- 多檔 tsc 輸出
- dist/*.mjs
- 含 dependencies
- 可供其他模型
- 非單一 IDE 外掛

8 Verify

- 參數不可臆造
- 依 Base URL 分流
- 非本地直接測
- 失敗才開本地站
- 可故意給錯值

授權分層：OS、API、MCP

OS

作業系統層

- 執行環境與邊界
- 程序隔離
- 誰能跑 MCP runtime
- 防止跨層操作

API

服務 API 層

- IP 白名單政策
- 業務身分權限
- 只暴露薄接口
- 不改 schema

MCP

MCP / AI HUB

- 工具清單=允許動作
- 參數可驗證
- Bearer + UserId
- 不繞過、不代操

必須手動呼叫 skill

Command template

/speckit-api-to-mcp 新增 API，提供...功能，可指定...作為參數，輸出...

1 Base URL

站台位置；沒給才從專案設定推斷

2 MCP port

預留 SSE；未指定預設 8895

3 Output path

指南與 MCP 同目錄 (MCP/)

4 Verify params

不可臆造；可故意給錯值

會議室已走過；帶走三項能力

AI HUB/Cursor MCP -> IP whitelist -> api/MeetingRoom* -> DB

查空房

查預約

我的預約

單筆預約

單筆取消

批次預約

批次取消

1 Skill craft

會寫、約束、審計；可重跑

2 MCP fit

何時掛、何時只要 API

3 Fast path

指南 -> 生成 -> dist -> 驗證

Fit

- 舊系統要給 Agent 呼叫
- 邏輯已在，只需標準接口
- 可攜專案且不覆蓋 constitution

Not a fit

- 要改表、大重構、從零做
- 只是改 UI / 內部 MVC
- 期望 Agent 自動觸發

11A | Ultimate Goal

把劣勢變為優勢

當系統、文件又大又多又雜——不是包裝，而是讓 Agent 反向利用這些「多」

1 反向利用大量程式

由 Agent 建立 Speckit 開發環境

程式越多越能推斷架構、慣例與邏輯，鑄造可重複的開發記憶。

2 Skill 審計最小化

新程式與既有系統的交互作用

先讀再寫、缺漏才補；用約束把變動鏈在最小範圍。

3 反向利用既存文件

統一規格與呼叫語言

文件越雜越需要 Agent 澄清、對齊，變成人與 Agent 都讀得懂的指南。

4 建立薄封裝 API

全面支援其他 Agent 呼叫

不重構舊系統；用標準接口 + MCP，讓多個 Agent 共用同一能力。

備註：MCP 傳輸模式 stdio vs SSE

stdio (首選)

特性

- 平台啟動子進程，stdin/stdout 通訊
- 不開網路埠；生命週期隨 Client
- 本專案預設；Cursor / AI HUB 同模型

建議場景

- AI HUB、Cursor 等支援 stdio 匯入
- 平台能 spawn 進程（同機/同信任邊界）
- 單 session 對應一 MCP；本機開發驗證

SSE / HTTP

特性

- 常駐 HTTP（預設 8895），URL 連線
- 可多客戶端共用同一實例
- 須維運埠、防火牆與連線安全

建議場景

- 平台不能/不願 spawn，只能連 URL
- 多客戶端需共用同一常駐 MCP
- 有 stdio 匯入時不用 SSE（多一層風險）

一句話

能 stdio 匯入（含 AI HUB）→ 用 stdio；只能連 URL、不能起子進程 → 才考慮 SSE。「遠端平台」不等於一定要 SSE。